

Java Programming 3

Week 9: JPA



Agenda this week

Last week - Project Review

What is ORM?

Spring configuration

The @Entity

CRUD operations with JPA

Implementing the repository

Relations with JPA

Last week - Project Review

- Last week review:
 - JDBC
- Project Remarks:
 - What went well - What went wrong?
 - Did you tag with *sprint3*?
- Repositories
 - Make a repository per main entity, do not make repositories for cross tables





Agenda this week

What is ORM?

Spring configuration

The @Entity

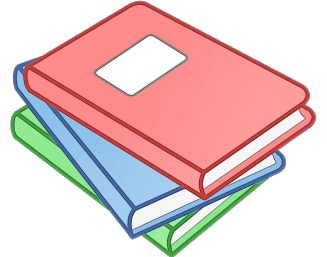
CRUD operations with JPA

Implementing the repository

Relations with JPA

Tutorials

- JPA WIKIBOOK: https://en.wikibooks.org/wiki/Java_Persistence
- JPQL examples: <https://docs.oracle.com/javaee/6/tutorial/doc/bnbt1.html>
- Relationships:
<https://stackabuse.com/a-guide-to-jpa-with-hibernate-relationship-mapping/>





Agenda this week

What is ORM?

Spring configuration

The @Entity

CRUD operations with JPA

Implementing the repository

Relations with JPA

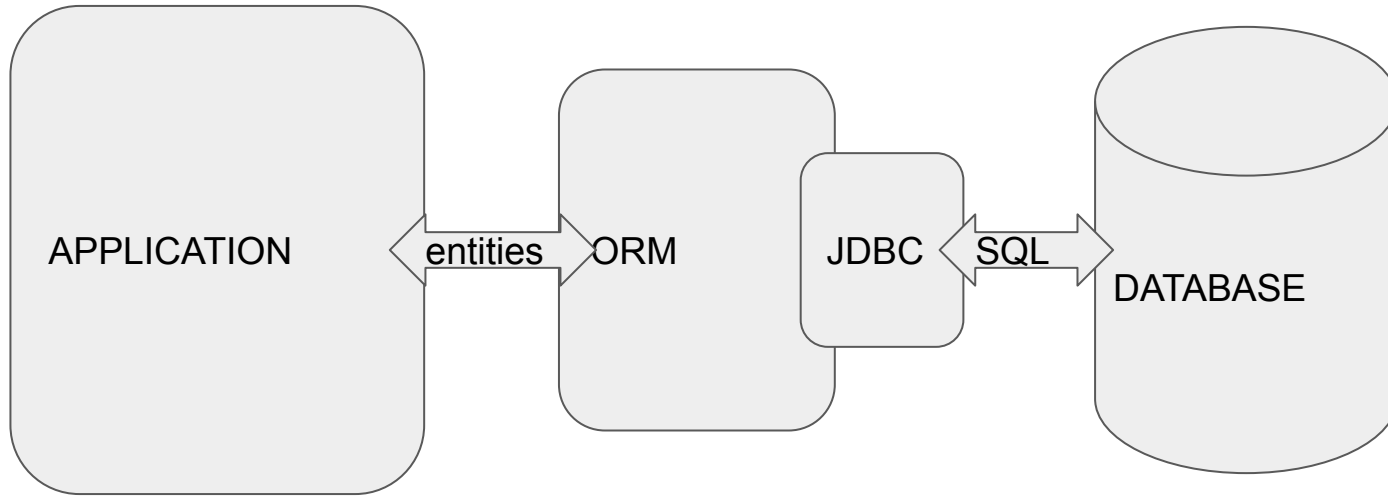
What is ORM?

- Object Relational Mapping
 - We use a framework that maps the objects from our domain model to tables in the relational database.
 - The application queries and updates the objects from the domain model instead of talking directly to the database
 - The ORM framework keeps the objects in sync with the database.
- JPA (“Jakarta Persistence API”) is a standard API defined for ORM
 - Hibernate provides an implementation of this standard.



HIBERNATE

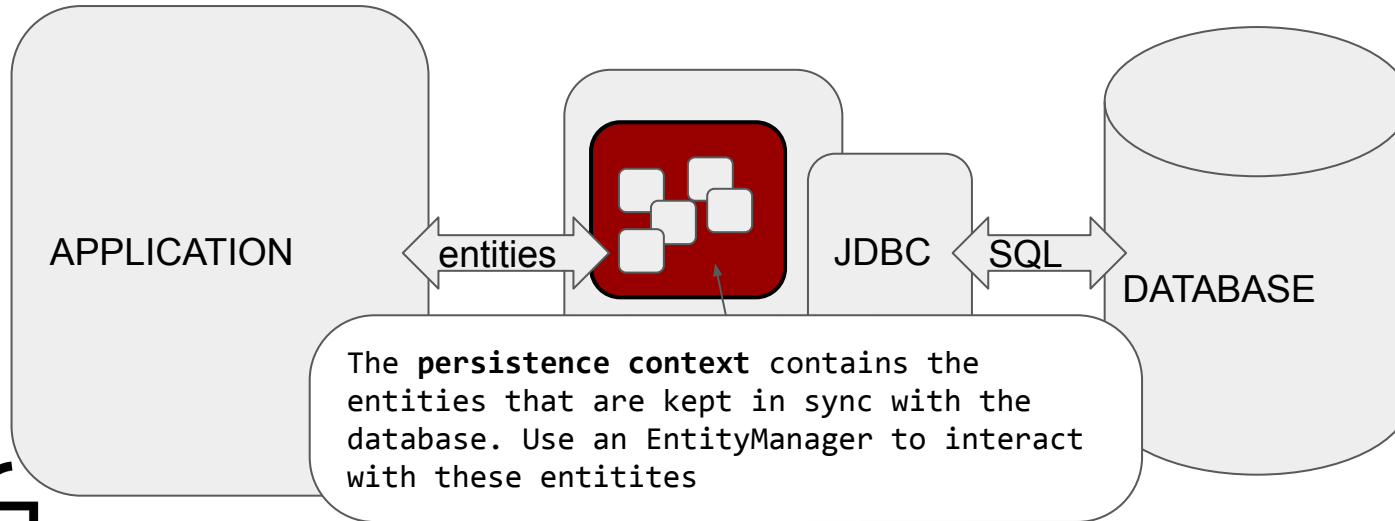
ORM



ORM Provides an extra abstraction layer on top of JDBC. The application uses the *entities* (often domain objects) to create, read, update and delete records from the database.

Persistence Context - EntityManager

- All the entities that are synced with the database reside in the *persistence context*
- Via an EntityManager you can manage the persistence context: you can create, remove, find and query entities.



Entities

- Domain objects that map to a database table get the `@Entity` annotation
- An entity needs
 - An `@Entity` class annotation
 - An id (`@Id`). It maps to the primary key of the table
 - A no arguments constructor (can be protected).

```
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String title;

    protected Book() {
    }
    //other constructors, getters, setters, toString, ...
}
```



Agenda this week

What is ORM?

Spring configuration

The @Entity

CRUD operations with JPA

Implementing the repository

Relations with JPA

How to configure Spring?

- Add the **Spring Data JPA** and **Postgresql** (or the DB you are using) driver dependencies
 - using spring initializr when creating the project
 - or by adding them to build.gradle.kts dependencies

```
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")  
    runtimeOnly("org.postgresql:postgresql")  
    testImplementation("org.springframework.boot:spring-boot-starter-test")  
}
```

How to configure Spring?

- Add these lines to the application.properties:

```
spring.datasource.url=jdbc:postgresql:pro3_db
spring.jpa.properties.hibernate.default_schema=demo_orm
spring.datasource.username=postgres
spring.datasource.password=Student_1234
spring.sql.init.mode=always
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
```

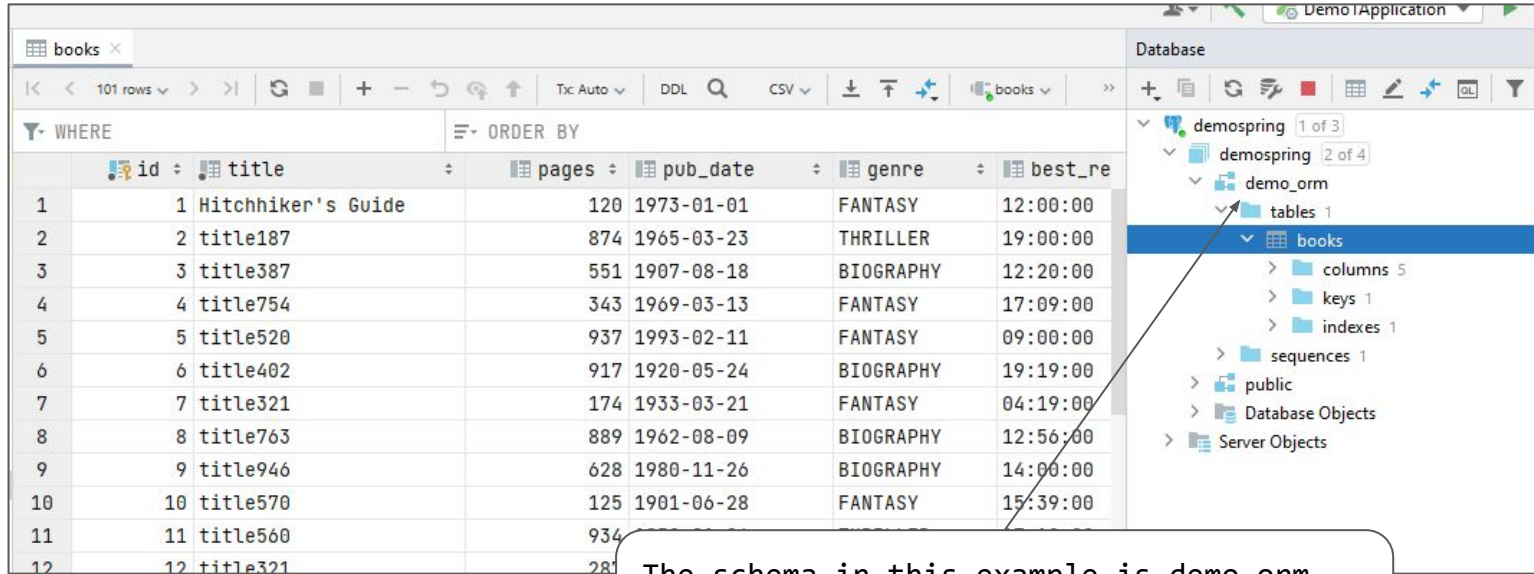
This is the datasource configuration. We are using Postgresql this time. I'm using a schema in the database. Create it in the database or adapt this to your Postgres configuration.

Will re-create the database tables **each time** you start the application. **Remove in production!!!**

This will show all executed SQL statements. Good for debugging!

- Some ddl-auto values:
 - create (default for in memory databases),
 - update (adds new columns or tables, but does not change existing),
 - none (default for external databases),
 - validate (error if database does not match ddl)

Use the IntelliJ database tool to inspect the database



The screenshot displays the IntelliJ IDEA Database tool window. The main pane shows a table named 'books' with the following data:

	id	title	pages	pub_date	genre	best_re
1	1	Hitchhiker's Guide	120	1973-01-01	FANTASY	12:00:00
2	2	title187	874	1965-03-23	THRILLER	19:00:00
3	3	title387	551	1907-08-18	BIOGRAPHY	12:20:00
4	4	title754	343	1969-03-13	FANTASY	17:09:00
5	5	title520	937	1993-02-11	FANTASY	09:00:00
6	6	title402	917	1920-05-24	BIOGRAPHY	19:19:00
7	7	title321	174	1933-03-21	FANTASY	04:19:00
8	8	title763	889	1962-08-09	BIOGRAPHY	12:56:00
9	9	title946	628	1980-11-26	BIOGRAPHY	14:00:00
10	10	title570	125	1901-06-28	FANTASY	15:39:00
11	11	title560	934			
12	12	title321	281			

The right sidebar shows the database structure. The 'demo_orm' schema is selected, and the 'books' table is highlighted under 'tables'.

The schema in this example is demo_orm

The EntityManagerFactory

- The create an EntityManager you need an EntityManagerFactory
- Spring injects it when you annotate it with @PersistenceUnit

```
@Component
public class DemoRunner implements CommandLineRunner {
    @PersistenceUnit
    private EntityManagerFactory entityManagerFactory;

    @Override
    public void run(String... args) throws Exception {
        EntityManager entityManager =
            entityManagerFactory.createEntityManager();
        //...use the entityManager to find entities etc...
    }
}
```



Agenda this week

What is ORM?

Spring configuration

The @Entity

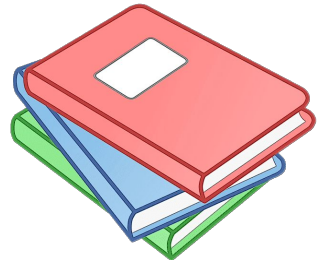
CRUD operations with JPA

Implementing the repository

Relations with JPA

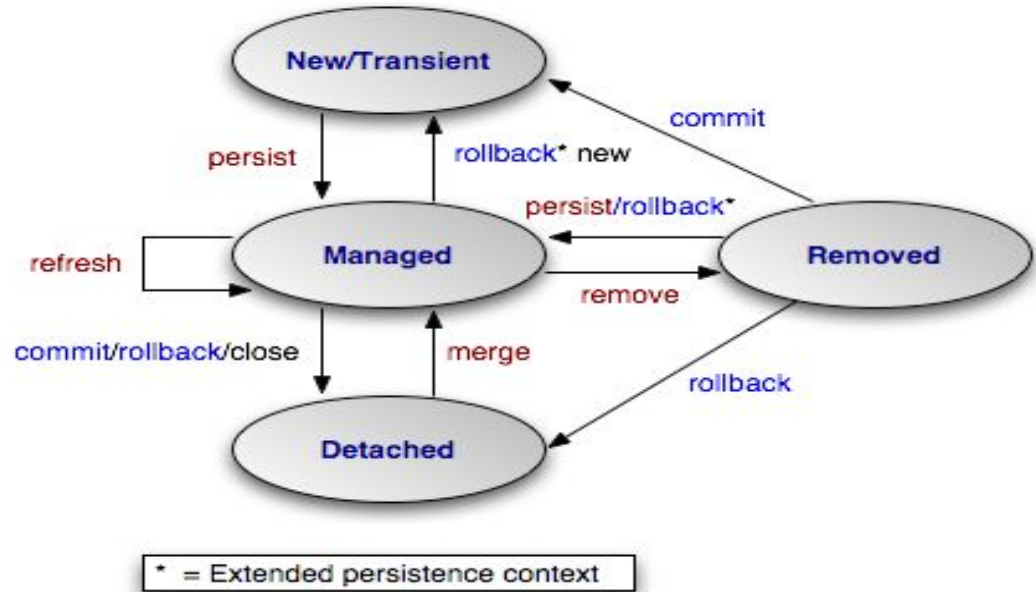
Entity details

- By default the entity maps to the table with the same name, you can change via `@Table(name="table_name")`
- By default the attributes map to the table column with the same name, you can change via `@Column(name="column_name")`
- For more details: <https://www.baeldung.com/jpa-entities>



Entity Lifecycle

- An entity can have different states: **New/Transient**, **Managed**, **Detached**, **Removed**
- **managed** entities will sync memory data changes to the database
- When the EntityManager is closed/committed the entity becomes *detached*.
- It can become *managed* again via the merge operation
- **Refresh** will reload the data from the database into the managed entities

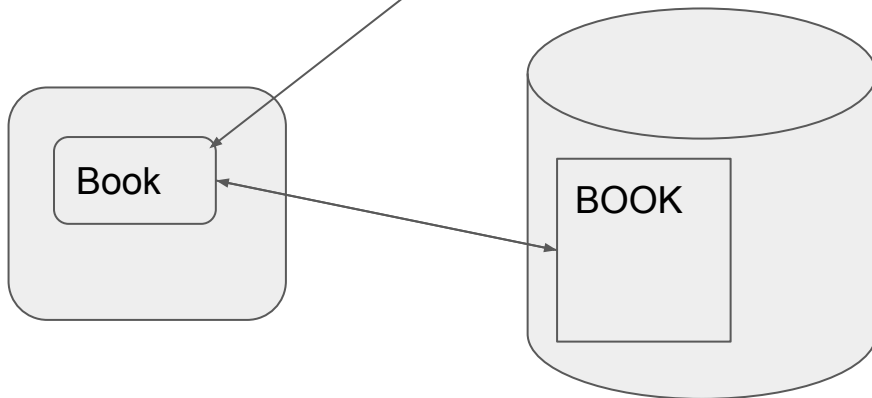


Example

Try with resources since JPA 3.1 /
Spring boot 3
For earlier versions call:
`entityManager.close();`

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    Book book = em.find(Book.class, 23);  
    book.setGenre(Genre.FANTASY);  
    em.getTransaction().commit();  
}
```

The book entity resides in the persistence context. It is in a managed state. Any change to it will be synced to the database when the transaction is committed.



Use the @Entity to make it an entity

- Use @GeneratedValue to tell JPA that the database will generate the key

```
@Entity  
@Table(name="books")  
public class Book {  
    @Id  
    @GeneratedValue(strategy = GenerationType. IDENTITY)  
    private int id;  
  
    protected Book() {}  
    //...
```

Entity is saved to table "books" (default is "book")

@Entity, @Id and no arguments constructor are always required

Exercise 1:

- create a Spring application with the Spring Data JPA and PostgreSQL dependencies
- configure it to use PostgreSQL
- inject an EntityManagerFactory using the @PersistenceUnit annotation
- create a Book entity class (with @Entity annotation)
- Start your application and inspect the hibernate SQL code



The example entity: Book

Book		
f	id	int
f	title	String
f	pages	int
f	datePublished	LocalDate
f	bestReadingTime	LocalTime
f	genre	Genre
m	Book()	
m	Book(String, int, LocalDate, LocalTime, Genre)	

Annotate the fields

- Use `@Column` to change the column name if it does not match the name of the attribute
 - by default Java camelCase is replaced by snake_case in the DB
- You can also specify properties of the column

```
@Entity
@Table(name="books")
public class Book {
    //...

    @Column(name = "book_title", nullable = false, length = 50)
    private String title;
    private int pages;

    //...
```

What about dates and times?

- LocalDate and LocalTime will be converted to SQL date and time...

```
@Entity
@Table(name="books")
public class Book {

    //...

    private LocalDate datePublished;

    private LocalTime bestReadingTime;

    //...
```

And enumerations?

- Use the `@Enumerated` annotation
- By default the ordinal is saved, but you can specify `EnumType.STRING` to save the value instead

```
public enum Genre {  
    THRILLER, BIOGRAPHY,  
    FANTASY  
}
```

```
@Entity  
@Table(name="books")  
public class Book {  
  
    //...  
  
    @Enumerated(EnumType.STRING)  
    private Genre genre;  
  
    //...
```

Resulting database table

- Because of the `spring.jpa.hibernate.ddl-auto=create` in the `application.properties`, the database is re-created each time.

```
-- auto-generated definition
CREATE TABLE books
(
    id                SERIAL
        CONSTRAINT books_pkey PRIMARY KEY,
    best_reading_time TIME,
    pub_date          DATE,
    genre              VARCHAR(255),
    pages             INTEGER,
    book_title        VARCHAR(50) NOT NULL
);
```

Exercise 2:

- Add fields to the book entity:
 - A title, which is not nullable and has max length of 50
 - A LocalDate field and LocalTime field
 - An enumeration
- Start the application and inspect the generated database table...





Agenda this week

What is ORM?

Spring configuration

The @Entity

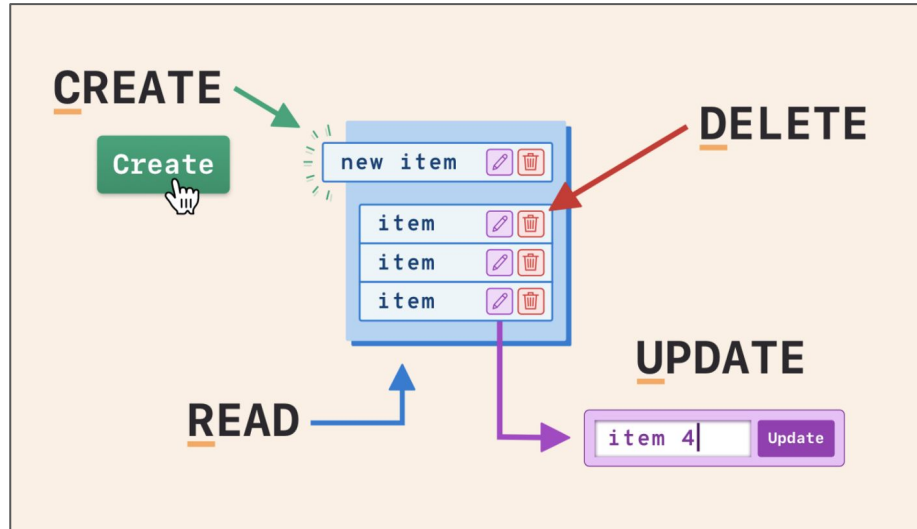
CRUD operations with JPA

Implementing the repository

Relations with JPA

JPA CRUD operations

- We will have a look at how the different CRUD operations are defined in JPA:
 - Creating and saving objects
 - Reading objects
 - Updating objects
 - Deleting objects



Create an entity and save to the database

```
Book book = new Book("Hitchhiker's Guide", 120,  
    LocalDate.of(1973, 1, 1),  
    LocalTime.NOON,  
    Genre.FANTASY);  
  
try(EntityManager entityManager =  
entityManagerFactory.createEntityManager()) {  
    entityManager.getTransaction().begin();  
    entityManager.persist(book);  
    entityManager.getTransaction().commit();  
}
```

All changes on the persistence context are done using transactions: you first begin the transaction, then perform the operations and finally commit (or rollback) the transaction.

The persist method inserts the entity into the database. Hibernate knows how to map it to the database using the annotations.

Primary key?

```
@Entity
@Table(name="books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    //...
```

We let the database generate the primary key values.

We can chose different strategies:

- GenerationType.AUTO
- GenerationType.IDENTITY
- GenerationType.UUID
- GenerationType.SEQUENCE
- GenerationType.TABLE

It depends on the database system but we will mostly use GenerationType.IDENTITY

The book has a key now!

```
Book book = new Book("Hitchhiker's Guide", 120,  
    LocalDate.of(1973, 1, 1),  
    LocalTime.NOON,  
    Genre.FANTASY);  
try(EntityManager entityManager = EntityManagerFactory.createEntityManager()) {  
    entityManager.getTransaction().begin();  
    entityManager.persist(book);  
    logger.info("> Created book:" + book);  
    entityManager.getTransaction().commit();  
}
```

The book is saved to the database, got an id there and the id is synced back to the entity...

```
> Created book:Book{id=1, title='Hitchhiker's Guide', pages=120, ...
```

Add 100 objects to the database...

```
try (EntityManager entityManager = entityManagerFactory.createEntityManager())
{
    entityManager.getTransaction().begin();
    Stream.generate(Book::randomBook)
        .limit(100)
        .forEach(entityManager::persist);
    entityManager.getTransaction().commit();
}
```

This code will add 100 random books to the database. I used a factory method in my entity class called randomBook to generate a random book.

```
public static Book randomBook(){
    Random random = new Random();
    return new Book("title"+random.nextInt(),
        random.nextInt(1000),
        LocalDate.ofEpochDay(random.nextLong(-10000,10000)),
        LocalTime.ofNanoOfDay(random.nextLong(24L*60*60*1000000000)),
        Genre.values()[random.nextInt(Genre.values().length)]);
}
```

Exercise 3:

- Create a static factory to generate random books
- Use the EntityManager to save some books to the database
- Inspect the data using the Database tool
- Is it possible to load initial data?
 - Yes, you can still use data.sql (and schema.sql)!
 - But you need to set (in application.properties):

```
spring.sql.init.mode=always
```

```
spring.jpa.defer-datasource-initialization=true
```

Wait until JPA
created your tables

- Load some initial Books into the database using data.sql...



Retrieve an object from the database

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    //em.getTransaction().begin();  
    book = em.find(Book.class, 23);  
    logger.log("Found book:" + book);  
    //em.getTransaction().commit();  
}
```

We can retrieve an object by it's key.
If the object is not found, this will
return null

If we only retrieve data from the
database, no transactions are needed!

Query entities from the persistence context

- You can use JPQL (JPA Query Language) to perform queries

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    List books = em.createQuery(  
        "SELECT b FROM Book b WHERE b.title LIKE '%the%'"  
    ).getResultList()  
    .forEach(System.out::println);  
}
```

- JPQL is similar to SQL, but:
 - you querying Java objects, not DB tables! Use Java attribute names (not DB columns)...
- You can even chain single valued attributes (no need to JOIN!), e.g.

Query with parameters

- You can use JPQL (JPA Query Language) to perform queries

```
try (em = entityManagerFactory.createEntityManager()) {  
    TypedQuery<Book> query = em.createQuery(  
        "select b from Book b where b.title = :title",  
        Book.class);  
    query.setParameter("title", "Hitchhiker's Guide");  
    List<Book> books = query.getResultList();  
    books.forEach(System.out::println);  
}
```

Use a parameter, similar to PreparedStatements in JDBC...

Optional second parameter informs the compiler this returns a List<Book>

Exercise 4:

- Check the JPA Query Language examples on
 - https://www.tutorialspoint.com/jpa/jpa_ipql.htm
 - <https://docs.oracle.com/javaee/6/tutorial/doc/bnbt1.html>
- Now try to perform some queries on your books:
 - Retrieve all thrillers
 - Show all books ordered by date published
 - Show all books published before 1985
 - Find book(s) with most pages
 - ...



Update a *managed* object in the database

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    book = em.find(Book.class, 23);  
    //book is still managed by the EntityManager!  
    book.setGenre(Genre.FANTASY);  
    em.getTransaction().commit();  
}
```

While the transaction is still open, the object is *managed* by the EntityManager. We can use setters to change the object. If we commit the transaction, the change will be saved to the database!

Update a *detached* object in the database

```
Book book;  
try (EntityManager em = entityManagerFactory.createEntityManager()) {  
    book = em.find(Book.class, 5);  
}  
book.setTitle("Updated title...");  
try (EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    em.merge(book) ;  
    em.getTransaction().commit();  
}
```

The EntityManager is closed, so the book is no longer managed. We call it *detached*...

If we want to update a detached book, we use the merge operation!
If the book does not exist in the database it is inserted.

Delete a *managed* book: use the remove method

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    book = em.find(Book.class, 23);  
    em.remove(book);  
    em.getTransaction().commit();  
}
```

JPA targets managing data as objects.

We need to find a book object, before we remove it. After the commit, the book is removed from the database.

You can also bypass JPA managed objects and use a delete (or update) JPQL statement to affect a lot of students DB records in bulk.

Examples of JPQL statements: <https://docs.oracle.com/javaee/6/tutorial/doc/bnbt1.html>

Delete a *detached* book: in the transaction merge (or find) first!

```
Book book;
try (EntityManager em = entityManagerFactory.createEntityManager()) {
    book = em.find(Book.class, 5);
}
try (EntityManager em = entityManagerFactory.createEntityManager()) {
    em.getTransaction().begin();
    book = em.merge(book);
    em.remove(book);
    em.getTransaction().commit();
}
```

Exercise 5:

- Perform the different update and delete operations in a CommandLineRunner
 - Update the book with id 5: change the title...
 - Update all thrillers: set the title to uppercase
 - Delete book with id 7
 - Delete all books that have best reading time before 12:00





Agenda this week

What is ORM?

Spring configuration

The @Entity

CRUD operations with JPA

Implementing the repository

Relations with JPA

Let's implement a BookRepository with JPA...

```
public interface BookRepository {  
    List<Book> findAll();  
    Book findById(int id);  
    Book create(Book book);  
    void update(Book book);  
    void delete(Book book);  
}
```

We start from this interface...

Let's implement a BookRepository with JPA...

```
@Repository
public class JPABookRepository implements BookRepository{
    @PersistenceUnit
    private EntityManagerFactory entityManagerFactory;

    @Override
    public List<Book> findAll() {
        try (EntityManager em = entityManagerFactory.createEntityManager()) {
            List<Book> books = em.createQuery("select b from Book b").getResultList();
            return books;
        }
    }

    @Override
    public Book findById(int id) {
        try (EntityManager em = entityManagerFactory.createEntityManager()) {
            Book book = em.find(Book.class, id);
            return book;
        }
    }

    //...
```

Always the same code. Can Spring help us?

Inject an EntityManager!

```
@Repository
public class JPABookRepository implements BookRepository{
    @PersistenceContext
    private EntityManager em;

    @Override
    public List<Book> findAll() {
        List<Book> books = em.createQuery("select b from Book b").getResultList();
        return books;
    }

    @Override
    public Book findById(int id) {
        Book book = em.find(Book.class, id);
        return book;
    }

    //...
```

Spring creates the EntityManager for us!

Let's implement a BookRepository with JPA...

```
@Repository
public class JpaBookRepository implements BookRepository {
    //...

    @Override
    public Book create(Book book) {
        em.getTransaction().begin();
        em.persist(book);
        em.getTransaction().commit();
        return book;
    }

    @Override
    public void update(Book book) {
        em.getTransaction().begin();
        em.merge(book);
        em.getTransaction().commit();
    }

    @Override
    public void delete(Book book) {
        //...
    }
}
```

But... this will not work,
you cannot manage your own
transactions on a shared entity
Manager!

You can use @Transactional

```
@Repository
public class JpaBookRepository implements BookRepository {
    //...

    @Override
    @Transactional
    public Book create(Book book) {
        em.persist(book);
        return book;
    }

    @Override
    @Transactional
    public void update(Book book) {
        book = em.merge(book);
    }

    @Override
    @Transactional
    public void delete(Book book) {
        //...
    }
}
```

Use @Transactional: spring will add the transaction logic (begin and commit of the transaction) on the injected EntityManager

@Transaction in Service Layer

- A service method contains business logic
- This can involve multiple operations on one or more repositories that should be executed together (in one transaction)
- Transaction demarcation is typically done in the Service layer
- **If your transaction boundaries are the same as the begin and start of the method** you can use `@Transactional` annotation on your Service methods
 - Spring will ensure that these steps are done in one transaction: this is the best place to use `@Transactional`
 - All entities created and loaded are part of the same persistence context
 - you will be able to do lazy loading in the service layer
- You can also use transactions in methods that only read data, to make lazy loading work in a service, in this case annotate with `@Transactional(readonly=true)`
- You can use an injected `EntityManager` in your Repositories now

@Transaction in Service Layer

- An example: Student and Book
 - Student has a List of Books (1-to-many relation)
 - The Books are lazy fetched (configured in the Student entity)
 - Presentation layer needs the books of a certain student

In presentation layer:

```
student = studentService.findStudent(1).get();  
System.out.println(student);  
System.out.println(student.getBooks());
```

This will not work, because the books are lazy fetched!

Exercise 6:

- Try to implement the BookRepository using JPA!
 - Start from the code on gitlab:
<https://gitlab.com/kdg-ti/programming-3/exercises/jpabookrepository>
 - Run it: it is a small console application to list, insert, update and delete books
 - Inspect the code:
 - it is 3-layered: presentation - service - repository
 - It uses H2 database
 - Implement the repository using JPA
 - Implement the service layer: delegate to the correct JPA methods
 - For the deleteBook you need 2 steps:
 - Find the book and delete it → **use @Transactional in your service layer!**
 - Implement the presentation layer





Agenda this week

What is ORM?

Spring configuration

The @Entity

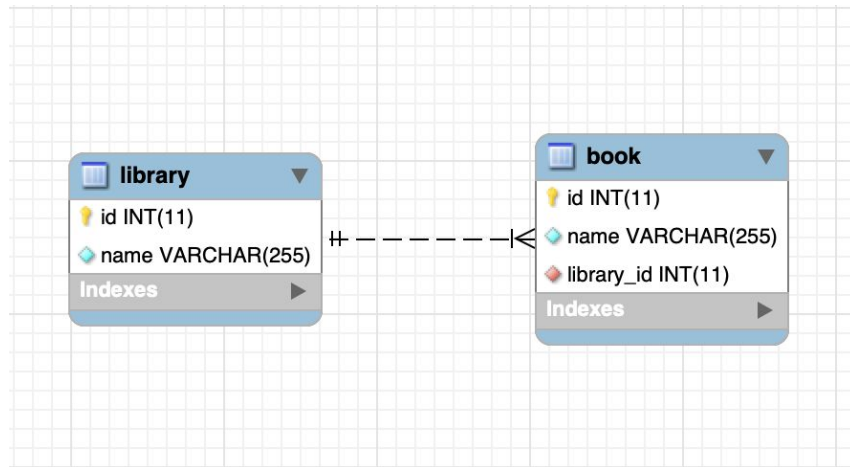
CRUD operations with JPA

Implementing the repository

Relations with JPA

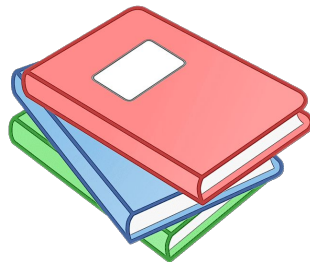
Relationship mappings

- What if we have
 - more than one table
 - Relationships between those tables?
- JPA has annotations for the different kinds of relationships:
 - One-to-One
 - One-to-Many and Many-to-One
 - Many-To-Many
- We will have a look at all of them...



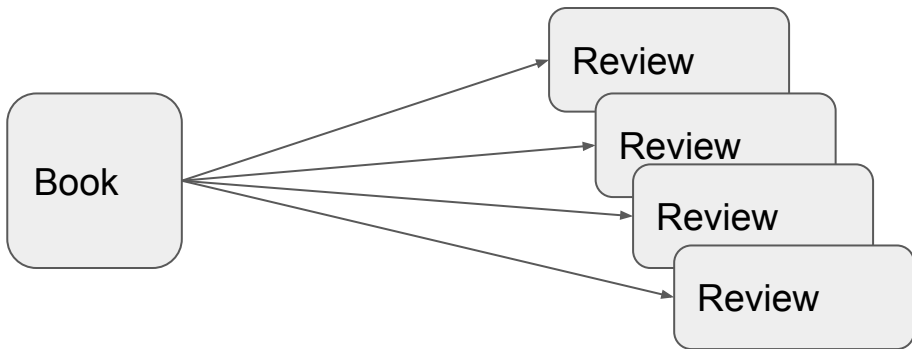
Tutorial on mappings

- <https://stackabuse.com/a-guide-to-jpa-with-hibernate-relationship-mapping/>
- Take the time to read through this tutorial!
- In what follows we will demonstrate all relation types in the following demo application:
 - <https://gitlab.com/kdg-ti/programming-3/demos/jpabookrepositorywithrelations>
→ make sure you understand this code!



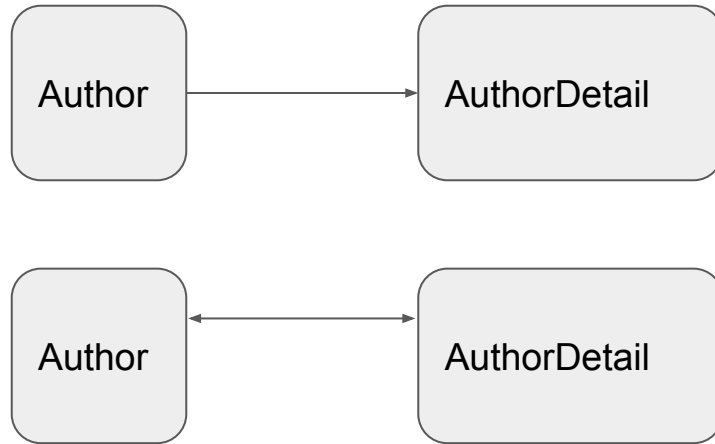
Fetch types: Eager versus Lazy Loading

- When we retrieve data: should we retrieve all related entities...?
 - **Eager** will retrieve everything
 - **Lazy** will only retrieve when it is accessed from code



Uni-directional versus Bi-directional

- Should we be able to get the Author via the AuthorDetail or not?



@OneToOne - uni-directional

- Example: **Author** (firstName, lastName) - AuthorDetails (email)

```
@Entity
@Table(name="authors")
public class Author {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    private String firstName;
    private String lastName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name="author_detail_id")
    private AuthorDetail authorDetail;
    //...getters and setters etc
}
```

We set the cascade type. If cascade is not specified, NO operations are cascaded.

@JoinColumn: only needed if you want to determine the name of the column with the FK in this table. author_detail_id is the default name (otherTablename_otherPKName)

@OneToOne - uni-directional

- Example: Author (firstName, lastName) - **AuthorDetails** (email)

```
@Entity
@Table(name="author details")
public class AuthorDetail {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    private String email;

    protected AuthorDetail() {
    }

    ///...
}
```

In the database

```
create table authors
(
    id                serial
        constraint authors_pkey
            primary key,
    first_name        varchar(255),
    last_name         varchar(255),
    author_detail_id integer
        constraint fk18v4mq362df6wh2xq5p60kvf7
        references author_details
);
```

```
create table author_details
(
    id                serial
        constraint author_details_pkey
            primary key,
    email             varchar(255)
);
```

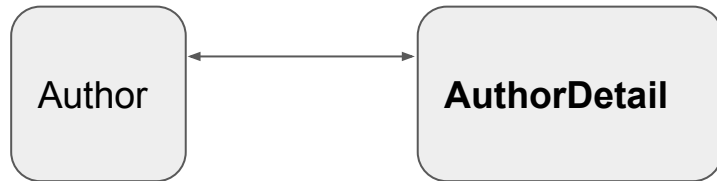
Using it in our Java code

- We create the 2 entities, link them together and persist the main entity
- Because of the cascade, the sub-entity will also be saved!

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    Author author = new Author("Herman", "Brusselmans");  
    author.setAuthorDetail(new AuthorDetail("hman@brussel.be"));  
    em.getTransaction().begin();  
    em.persist(author);  
    em.getTransaction().commit();  
}  
//...
```

If you remove an author, the
authordetails will also be
deleted!

OneToOne - bi-directional



- We only need to add the reference back from AuthorDetail...

```
@Entity
@Table(name="author_details")
public class AuthorDetail {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    private String email;

    @OneToOne(mappedBy = "authorDetail")
    private Author author;

    //... getters and setters etc..
}
```

The mappedBy refers to the authorDetail field of the Author class.

mapped By indicates this is the **inverse side**. The other side (owning side) specifies the relation characteristics (JoinColumns...) and by default has the foreign key.

If you do NOT specify mappedBy you will have TWO unidirectional OneToOne relations with an FK in each table!

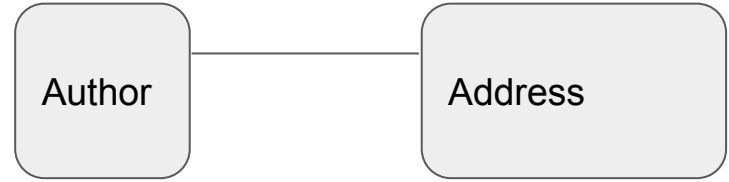
Using it in our Java code

- We load AuthorDetails with id 1
- We can get the Author and show it

```
try (EntityManager em = entityManagerFactory.createEntityManager()) {  
    Author author = em.find(Author.class, 1);  
    System.out.println("Lazy or Eager?");  
    System.out.println(author.getAuthorDetail());  
}
```

JPA uses Eager Loading: we don't see any sql query running after the "Lazy or Eager" print...

OneToOne - Single table



- All Address fields go into the Author Table

```
@Entity
@Table(name="authors")
public class Author {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    private String firstName;
    private String lastName;
    @Embedded
    private Address address;
    //...
}
```

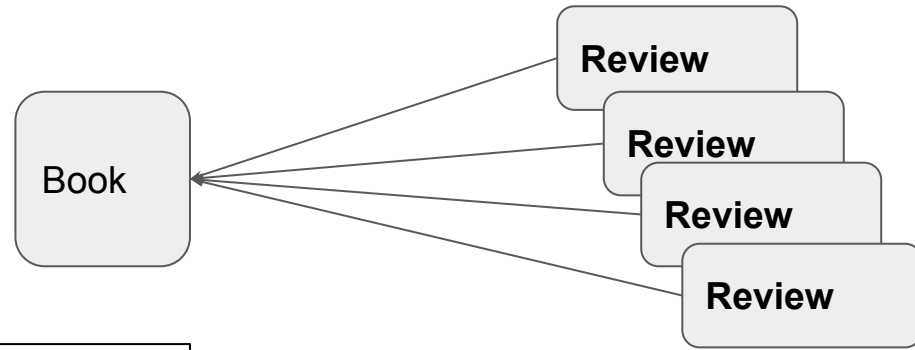
```
@Embeddable
public class Address {
    private String street;
    private String number;
    private String zip;
    private String community;
    //...
}
```

Specifying one of @Embeddable OR @Embedded is fine for embedding the fields of Address into the authors table.

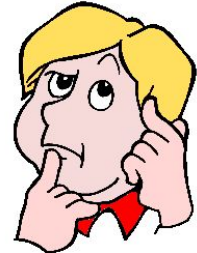
ManyToOne Uni-directional

- In **Review**:

```
//...  
@ManyToOne(cascade = { CascadeType.MERGE,  
    CascadeType.PERSIST, CascadeType.REFRESH})  
private Book book;  
//...
```



We cascade everything, **except** REMOVE!
Good idea?

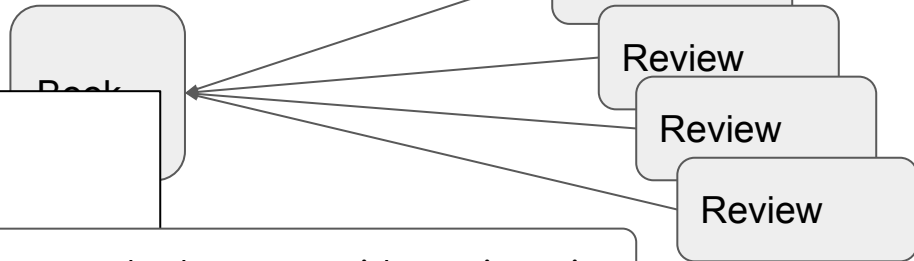


In the database

```
create table books
(
  id          serial
    constraint books_pkey
      primary key,
  best_reading_time time,
  pub_date     date,
  genre        varchar(255),
  pages        integer,
  title        varchar(50)
);
```

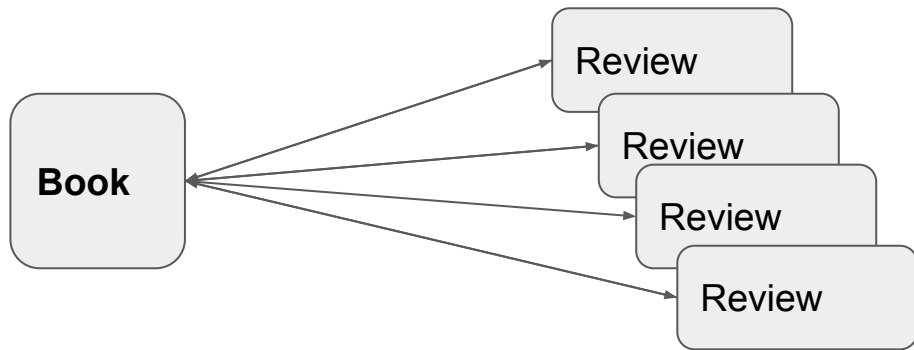
No changes to books, Many side reviews is the owning side and has foreign key.

```
create table reviews
(
  id          serial
    constraint review_pkey
      primary key,
  contents    varchar(255),
  book id     integer
    constraint fk880c0rw9ffetwe2p1qb7x4icf
      references books
);
```



ManyToOne/OneToMany bi-directional

- We'll add code for the other direction
- In Book add:



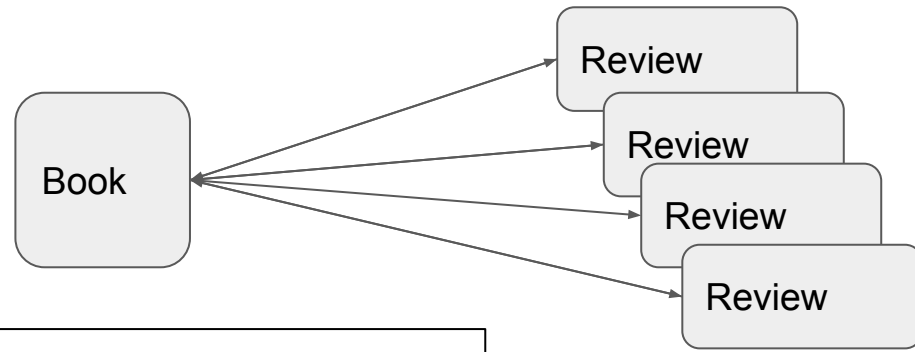
```
//...  
@OneToMany(mappedBy = "book", cascade = CascadeType.ALL)  
private List<Review> reviews = new ArrayList<>();  
  
//...
```

Book contains a List of reviews...

If you don't need the reviews most of the time, just leave the relation unidirectional and retrieve the reviews from the database when you need them.

OneToMany - bi-directional

- In Book:



```
//...  
public List<Review> getReviews() { return reviews;}  
  
public void setReviews(List<Review> reviews) {  
    this.reviews = reviews;  
    for (Review review : reviews) review.setBook(this);  
}  
  
public void addReview(Review review){  
    reviews.add(review);  
    review.setBook(this);  
}
```

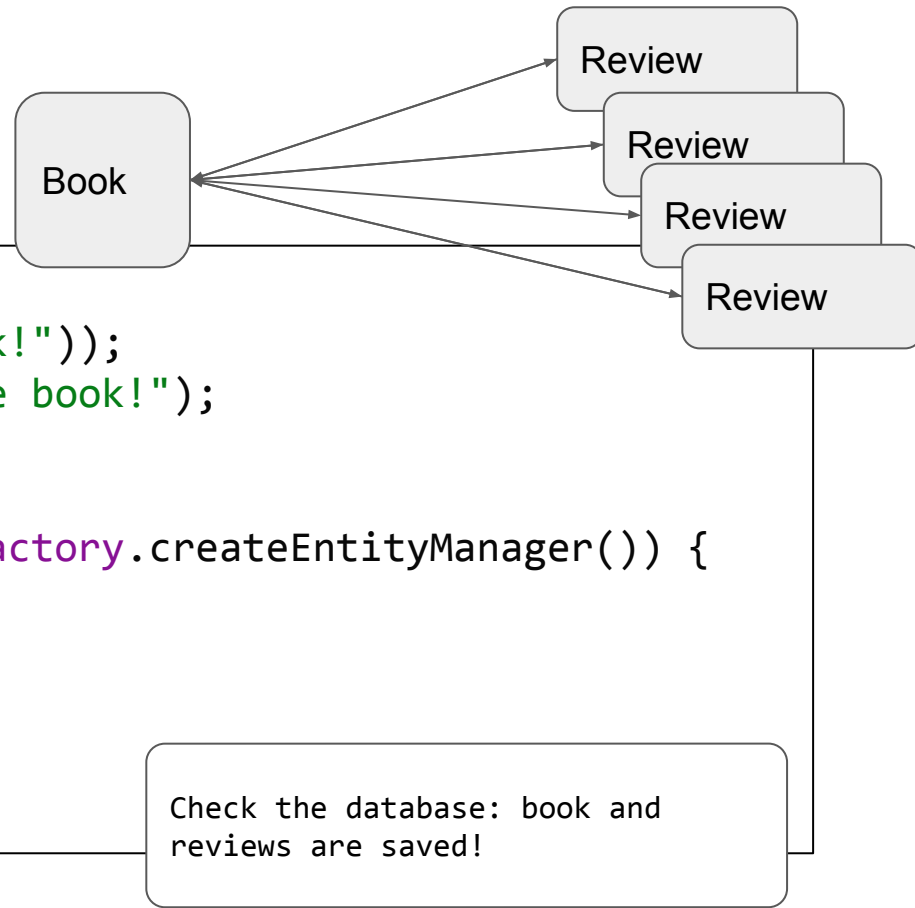
We add getter, setter and addreview method

Using it in our Java code

```
Book book = Book.randomBook();  
book.addReview( new Review("Great book!"));  
Review horrible = new Review("Horrible book!");  
book.addReview(horrible);
```

```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    em.persist(book);  
    em.getTransaction().commit();  
}
```

Check the database: book and reviews are saved!



Exercise 7:

- Implement `Book::removeReview(Review review)`
 - make sure review has an equals method
- Remove the review in the horrible variable from the Book and save the change to the DB
- Retrieve the book from the DB after doing so and look at the result (or use postgres and look in the table after running)



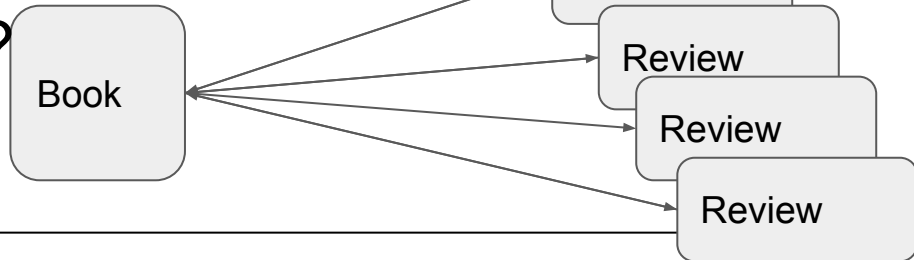
Exercise 7 part 2

- Did you succeed? Here are some hints:
 - This is a bidirectional relationship (removeReview should update both sides)
 - The Review is the owning side
- Now add code to print all reviews: The book in the horrible review should be null. But does that make sense?
- If you want a review to be DELETED from the DB when it is removed from the list of reviews of the book, modify the relation in Book:

```
@OneToMany(mappedBy = "book", cascade = CascadeType.ALL, fetch =  
FetchType.EAGER, orphanRemoval = true)  
private List<Review> reviews = new ArrayList<>();
```



Are we Eager or Lazy loading?



```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    book = em.find(Book.class, 1);  
    System.out.println("Loading the reviews...");  
    List<Review> reviews = book.getReviews();  
    reviews.forEach(System.out::println);  
}
```

Hibernate performs a select after the "Loading the reviews": it uses lazy loading!

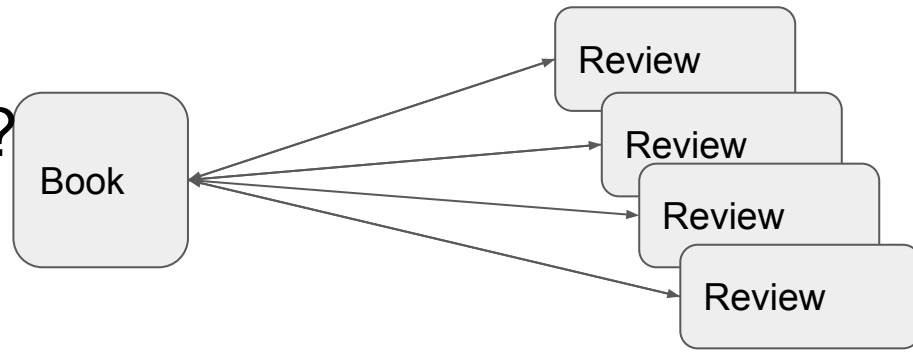
Loading the reviews...

Hibernate: select reviews0_.book_id as book_id3_3_0_, reviews0_.id
Review{id=1, contents='Great book!'}
Review{id=2, contents='Horrible book!'}

Default fetch types are
LAZY for XxxToMany relations and
EAGER for XxxToOne relations

You can override the default fetchtype in the entity annotation. example:
`@OneToMany(fetch=FetchType.EAGER)`

Are we Eager or Lazy loading?

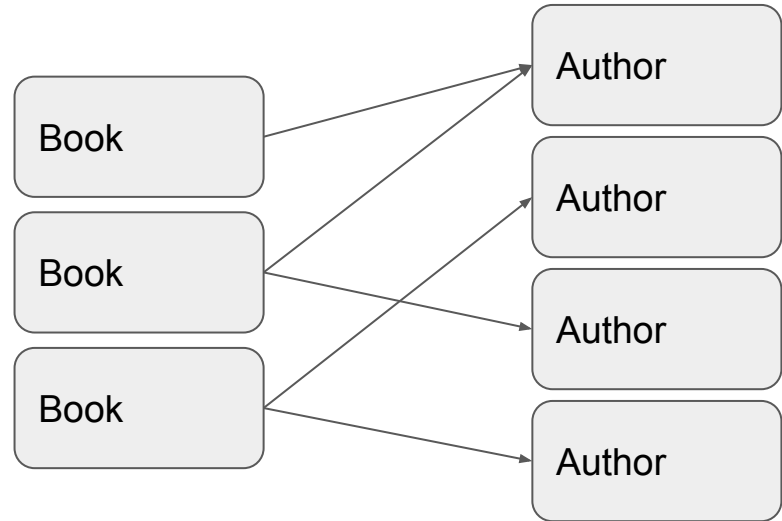


- We do not always need the reviews. What if we want to decide on a case by case basis?
 - Define the relation as LAZY
 - When you want to EAGER load, use a FETCH JOIN query

```
book = em.createQuery("""  
SELECT b FROM Book b  
JOIN FETCH b.reviews  
WHERE b.id = :id  
""", Book.class)  
    .setParameter("id", 1)  
    .getSingleResult();
```

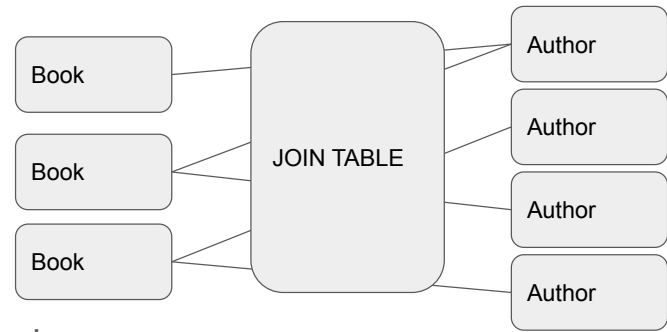
@ManyToMany - uni-directional

- A book can have more than one author
- If we delete a Book, we don't want to delete the Author



@ManyToMany - uni-directional

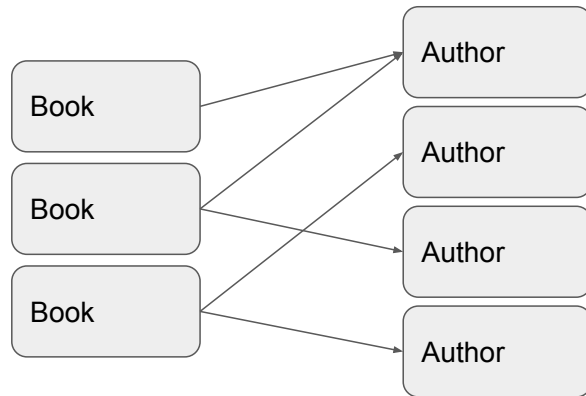
- We use a JOIN TABLE
 - Provides mapping between the two tables using 2 foreign keys



```
create table book_author
(
    book_id integer not null
        constraint
fk91ierknt446aaqnjl4uxjyls3
        references books,
    author_id integer not null
        constraint
fkro54jqpth9cqm1899dnuu9lqg
        references authors
);
```

@ManyToMany - uni-directional

- In class Book:

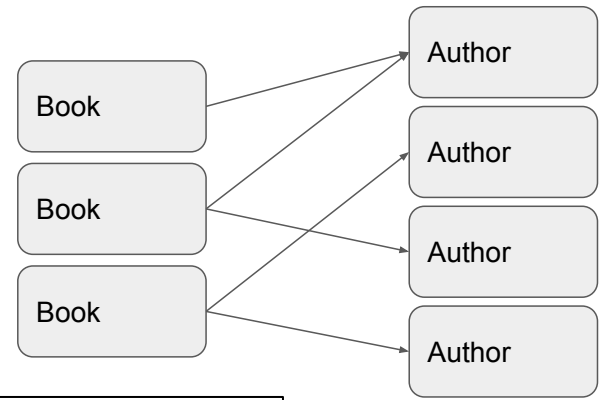


```
@ManyToMany(cascade = {CascadeType.DETACH, CascadeType.MERGE,  
    CascadeType.PERSIST, CascadeType.REFRESH})  
@JoinTable(name="book_author",  
    joinColumns = @JoinColumn(name = "book_id"),  
    inverseJoinColumns = @JoinColumn(name="author_id"))  
private List<Author> authors = new ArrayList<>();
```

And add getter and setter and addAuthor
method (addAuthor calls author.addBook)
...

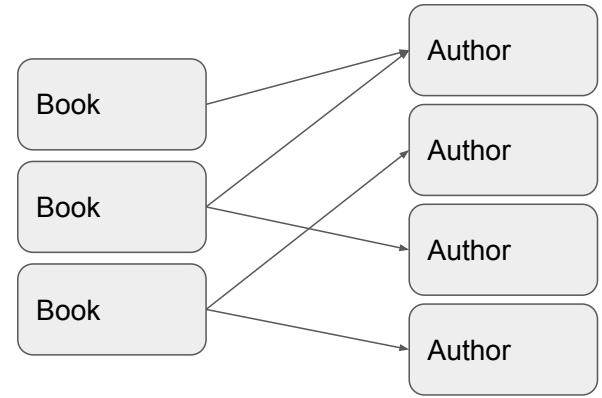
If you omit @JoinTable..., defaults are
table books_authors(books_id,authors_id)

Using it in our Java code



```
Book book1 = Book.randomBook();  
Book book2 = Book.randomBook();  
Book book3 = Book.randomBook();  
Author author1 = new Author("john", "beck");  
Author author2 = new Author("mario", "vargas");  
Author author3 = new Author("tine", "verstrepen");  
book1.addAuthor(author1);  
book1.addAuthor(author2);  
book2.addAuthor(author1);  
book3.addAuthor(author3);
```

Using it in our Java code

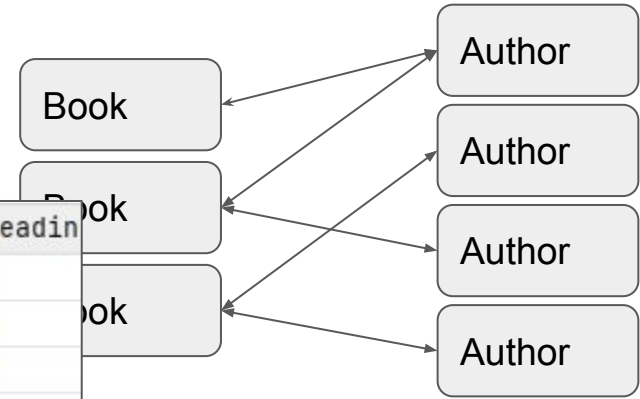


```
try(EntityManager em = entityManagerFactory.createEntityManager()) {  
    em.getTransaction().begin();  
    em.persist(book1);  
    em.persist(book2);  
    em.persist(book3);  
    em.getTransaction().commit();  
}
```

Check the database, do you find all the records?

Check the database...

id	title	pages	pub_date	genre	best_reading
1	title570	161	1990-02-10	FANTASY	19:46:00
2	title890	413	1995-05-11	THRILLER	16:06:00
3	title663	455	1975-04-03	FANTASY	06:40:00

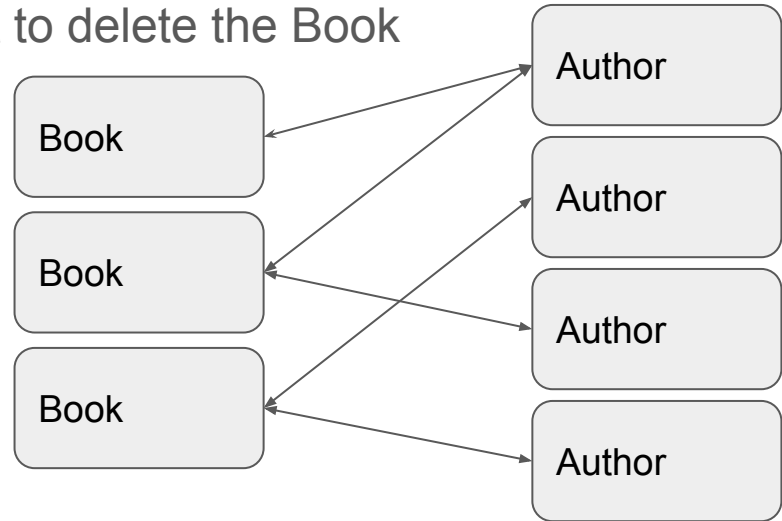


id	first_name	last_name	author_detail_id
1	john	beck	<null>
2	mario	vargas	<null>
3	tine	verstrepen	<null>

book_id	author_id
1	1
1	2
2	1
3	3

@ManyToMany - bi-directional

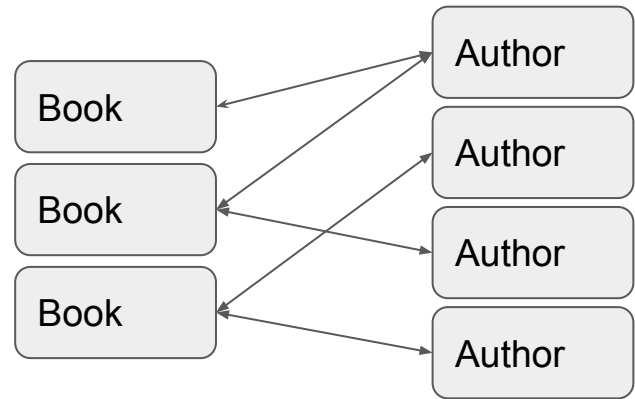
- A book can have more than one author
- an author can write more than one book
 - If you are not using both directions frequently, leave the relation unidirectional
- If we delete a Book, we don't want to delete the Author
- If we delete an Author, we don't want to delete the Book



@ManyToMany - bi-directional

- In class Author:

```
@ManyToMany(mappedBy = "authors", cascade =  
{CascadeType.DETACH, CascadeType.MERGE, CascadeType.PERSIST,  
CascadeType.REFRESH})  
private List<Book> books = new ArrayList<>();
```



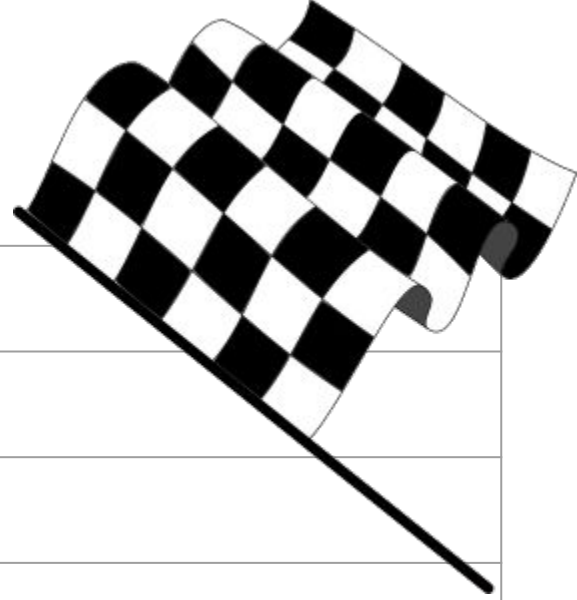
And add getter and setter and addBook method...
Guarantee that when an Author is added to a Book, the Book is also added to the author.

Exercise 8:

- Take the Student Management System example of last week (the code is in the exercises section on canvas)
- Now try to convert this demo to a JPA implementation



Agenda this week



What is ORM?
Spring configuration
The @Entity
CRUD operations with JPA
Implementing the repository
Relations with JPA

Project

- This week you will work out a third version of your repository layer: this time you will use JPA to implement the repository
- Add an extra profile to switch to this implementation
- Try to implement all the relationships between your entities
- Provide an application-dev.properties with H2 and application-prod.properties with Postgres configuration...

